

Container Orchestration with Kubernetes Activities

Table of Contents

Activity 1: Software setup	4
1.1. Prerequisite	4
1.2. Scenario	4
1.3. Steps	4
Activity 2: Setup kubectl	12
2.1. Prerequisite	12
2.2. Scenario	12
2.3. Steps	12
Activity 3: Setup minikube cluster	13
3.1. Prerequisite	13
3.2. Scenario	13
3.3. Steps	13
Activity 4: Kubernetes commands	15
4.1. Prerequisite	15
4.2. Scenario	15
4.3. Steps	15
Activity 5: Deploy Spring Boot Application	20
5.1. Prerequisite	20
5.2. Scenario	20
5.3. Steps	20
Activity 6: Scale spring boot application	24
6.1. Prerequisite	24
6.2. Scenario	24
6.3. Steps	24
Activity 7: Autoscaling spring boot application	26
7.1. Prerequisite	26
7.2. Scenario	26
7.3. Steps	26
Activity 8: Deploying Spring Boot Application using manifest file	27
8.1. Prerequisite	27
8.2. Scenario	27
8.3. Steps	27
Activity 9: Changing Deployment Image	32
9.1. Prerequisite	32

9.2.	Scenario	32
9.3.	Steps	32
Activity 10: Deploying Spring Boot App with MySQL		35
10.1.	Prerequisite	35
10.2.	Scenario	35
10.3.	Steps	35
Activity 11: Deploying Spring Boot app with CPU and Memory Limits		40
11.1.	Prerequisite	40
11.2.	Scenario	40
11.3.	Steps	40

Activity 1: Software setup

1.1. Prerequisite

Running virtual machine with ubuntu

1.2. Scenario

Installing docker, jdk17, maven on ubuntu

1.3. Steps

1

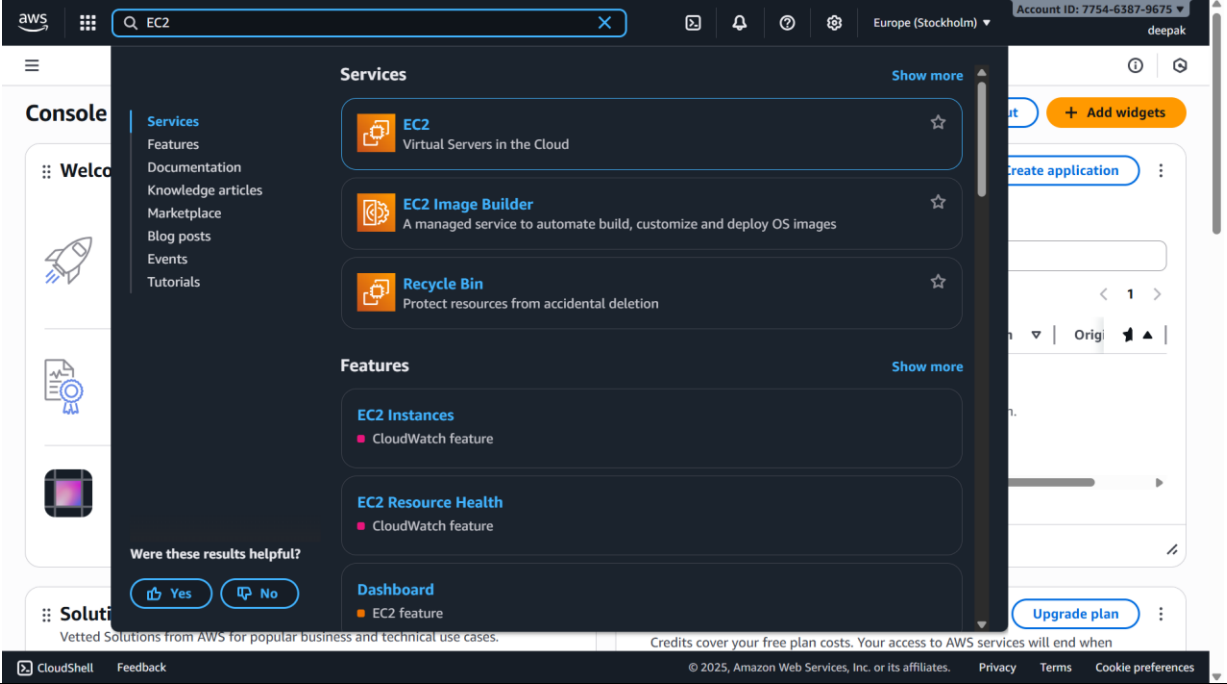
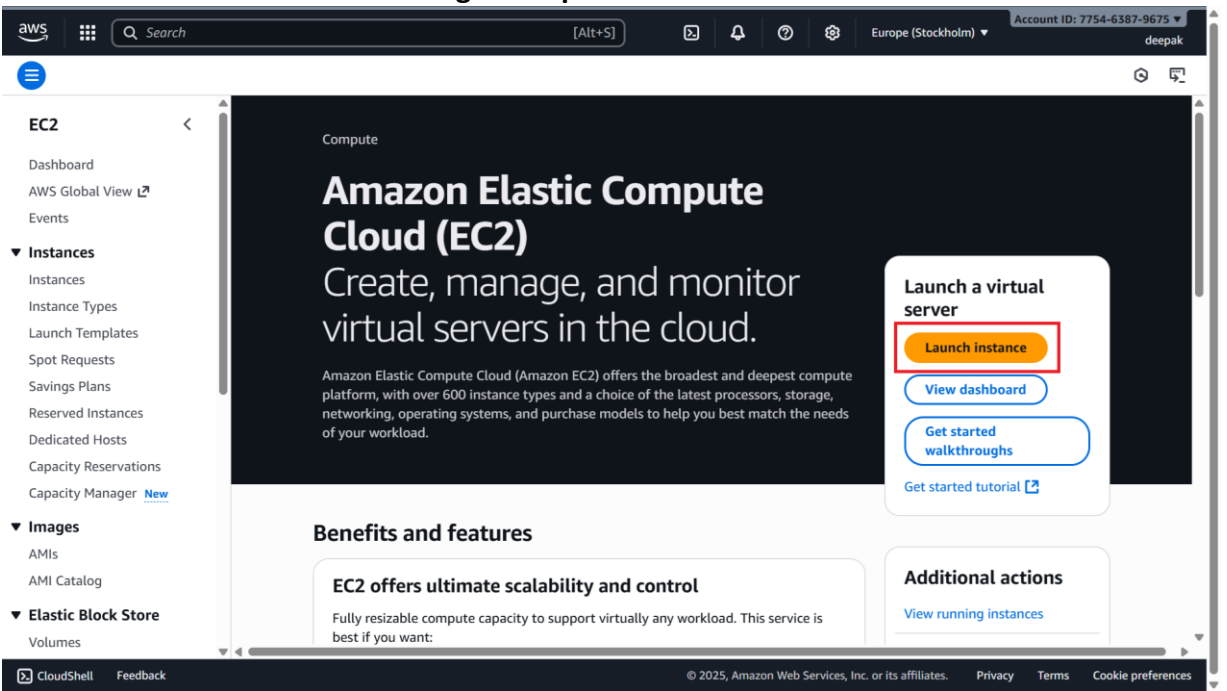
1. Log in to AWS Console

- Visit <https://aws.amazon.com/console>
- Sign in with your credentials

2

2. Navigate to EC2 Dashboard

- In the AWS Console, search for **EC2**

	
3	3. Click Launch Instance to begin setup 
4	3. Choose an Amazon Machine Image (AMI) • Select Ubuntu Server 22.04 LTS or Ubuntu 24.04 LTS (recommended for longer support) • Ensure it is marked Free Tier eligible

aws | Search | [Alt+S] | Europe (Stockholm) | Account ID: 7754-6387-9675 | deepak

EC2 > Instances > Launch an instance

Name and tags Info

Name: DevOps Add additional tags

Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recents Quick Start

Amazon Linux macOS **Ubuntu** Windows Red Hat SUSE Linu. Browse more AMIs

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type Free tier eligible

ami-0a716d3f3b16d290c (64-bit (x86)) / ami-0cab1941a8a08b817 (64-bit (Arm))

Virtualization: hvm ENA enabled: true Root device type: ebs

Summary

Number of instances Info: 1

Software Image (AMI): Canonical, Ubuntu, 24.04, amd64...read more

Virtual server type (instance type): t3.micro

Firewall (security group): New security group

Storage (volumes): 1 volume(s) - 8 GiB

Cancel Launch instance Preview code

CloudShell Feedback | © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

5

4. Select Instance Type

- Choose **t2.micro** (1 vCPU, 1 GB RAM) – Free Tier eligible

aws | Search | [Alt+S] | Europe (Stockholm) | Account ID: 7754-6387-9675 | deepak

EC2 > Instances > Launch an instance

Instance type Info Get advice

Instance type

t3.micro Free tier eligible

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Ubuntu Pro base pricing: 0.0143 USD per Hour

On-Demand RHEL base pricing: 0.0396 USD per Hour

On-Demand SUSE base pricing: 0.0108 USD per Hour

On-Demand Linux base pricing: 0.0108 USD per Hour

On-Demand Windows base pricing: 0.02 USD per Hour

Additional costs apply for AMIs with pre-installed software

All generations Compare instance types

Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key before you launch the instance.

Key pair name - required

Select Create new key pair

Network settings Info Edit

Summary

Number of instances Info: 1

Software Image (AMI): Canonical, Ubuntu, 24.04, amd64...read more

Virtual server type (instance type): t3.micro

Firewall (security group): New security group

Storage (volumes): 1 volume(s) - 8 GiB

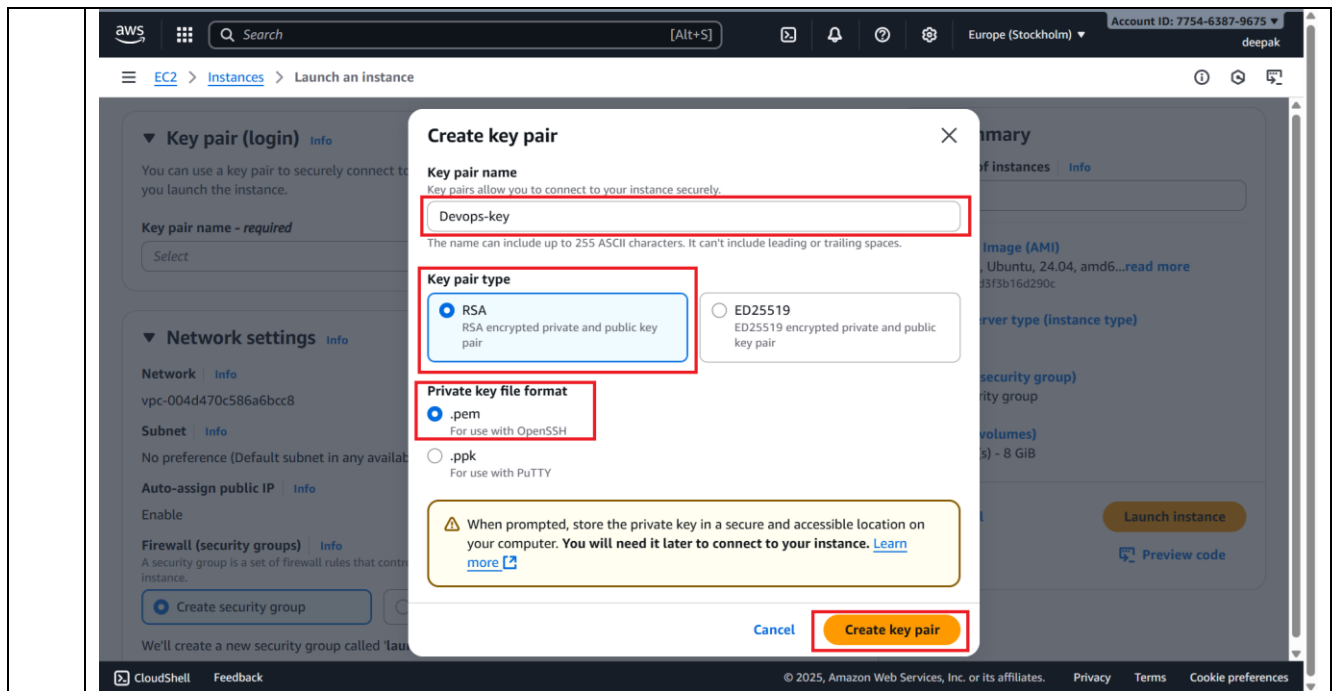
Cancel Launch instance Preview code

CloudShell Feedback | © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

6

Configure Key Pair

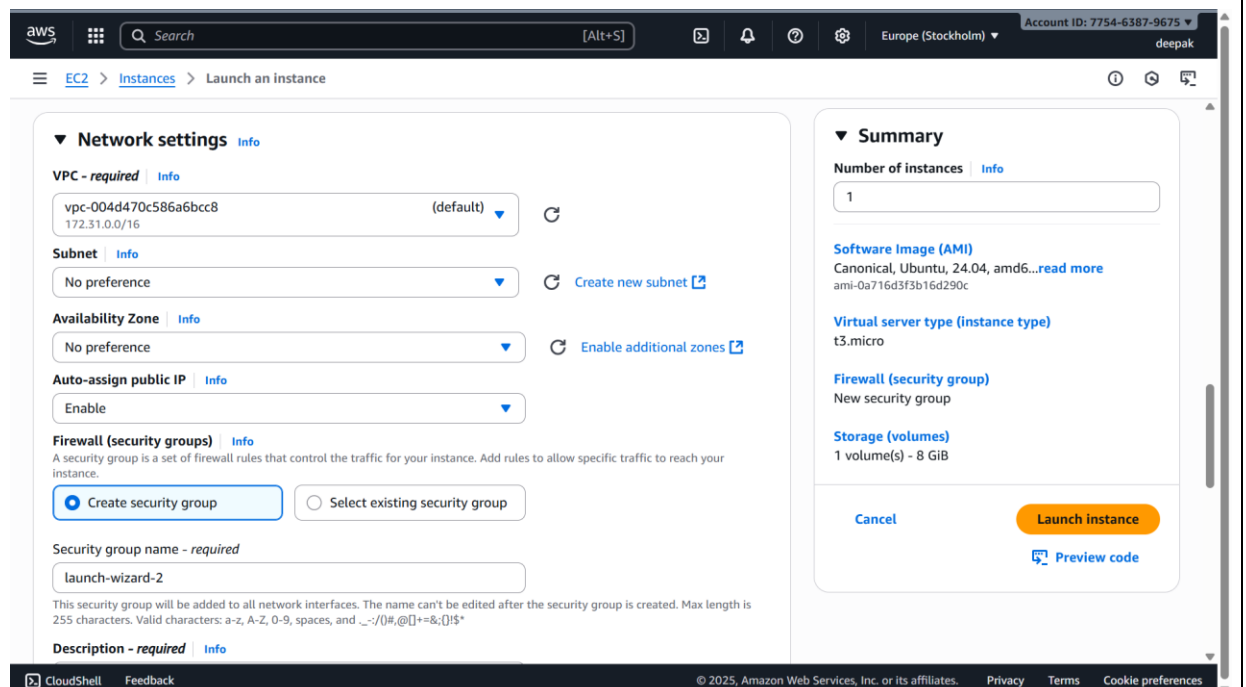
- Create a new key pair or use an existing one
- Choose .pem format for Linux/macOS or .ppk for Windows
- Download and store securely (you won't be able to download it again)

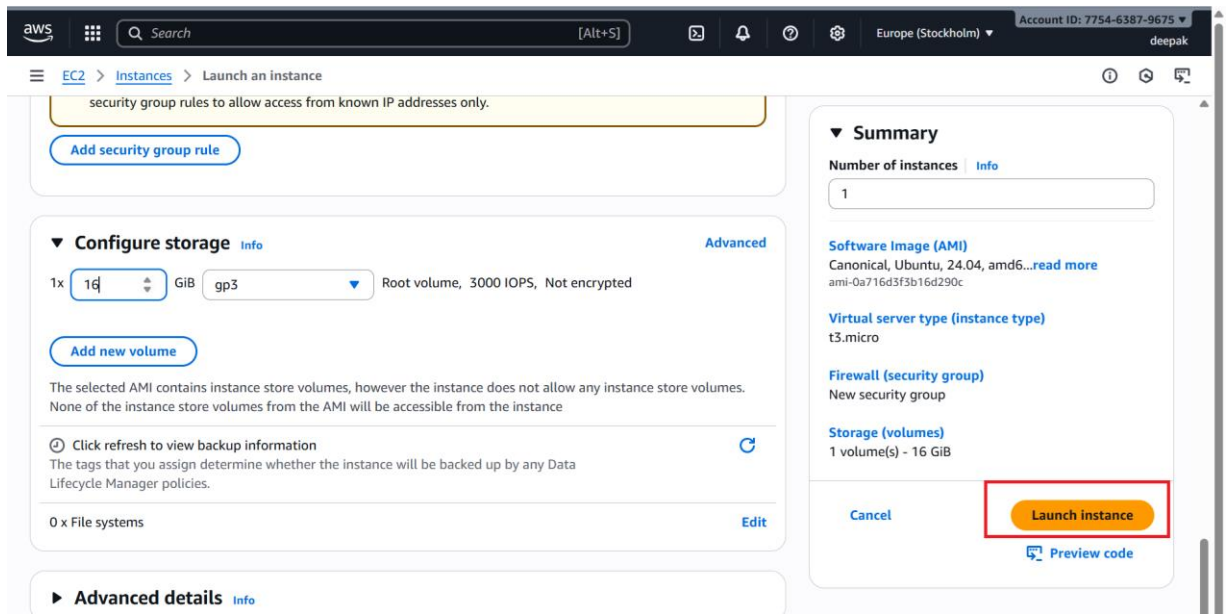


7

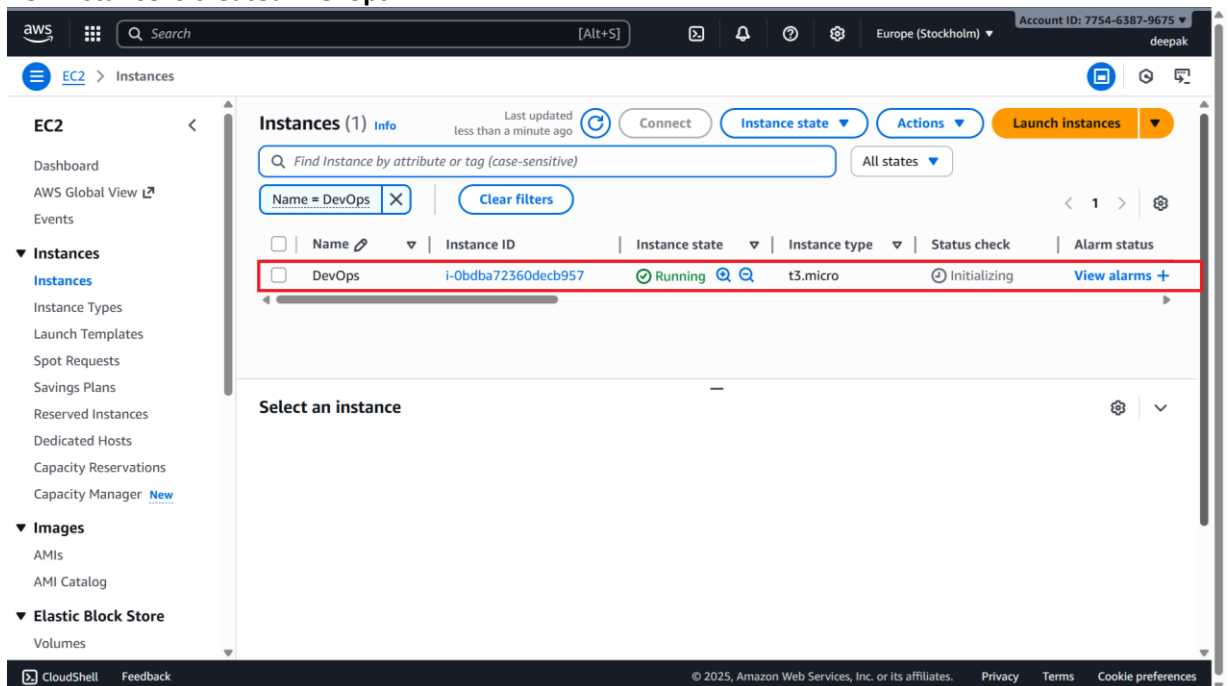
Configure Network Settings

- Use default VPC and subnet
- Create or select a **Security Group**
 - Allow **SSH (port 22)** from your IP
 - Optionally allow **HTTP (port 80)** and **HTTPS (port 443)** if hosting a web server





10 EC2 instance is created: DevOps



11 Connect to Your Instance

The screenshot shows the AWS Management Console for the EC2 service. In the left-hand navigation pane, the 'Instances' link is selected. The main content area displays a list of instances. The instance 'DevOps' with ID 'i-0bdba72360decb957' is highlighted. A red box is drawn around the 'Connect' button at the top of the instance list. Below the list, the details for the selected instance are shown, including its public IP address '16.171.177.177'.

12

Connect to Your Instance

The screenshot shows the 'Connect to instance' page in the AWS Management Console. The 'EC2 Instance Connect' tab is selected. Under the 'Connection type' section, the option 'Connect using a Public IP' is selected. The 'Public IPv4 address' is listed as '16.171.177.177'. The 'Username' field is filled with 'ubuntu'. A red box is drawn around the 'Connect' button at the bottom right of the page.

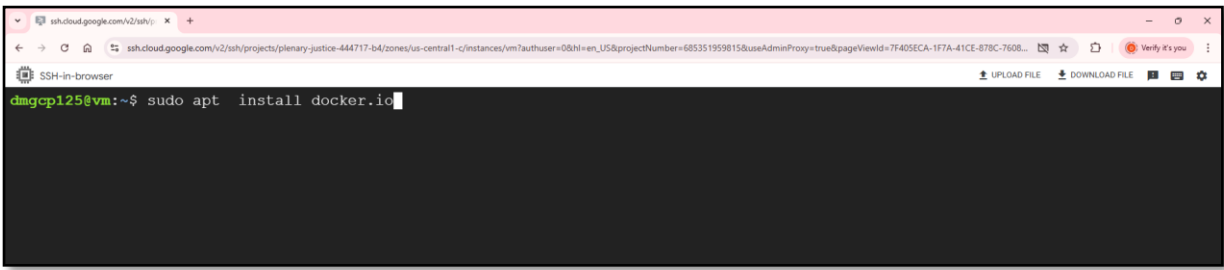
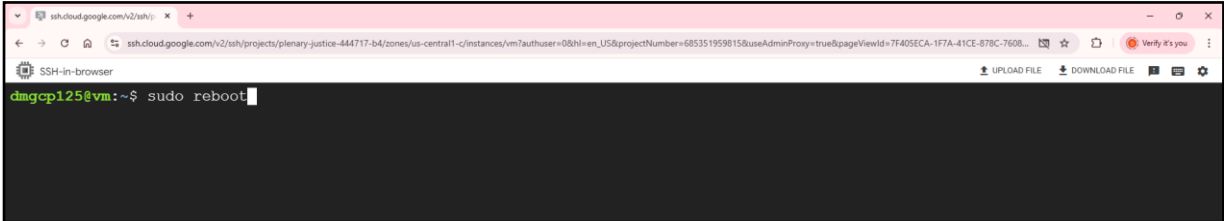
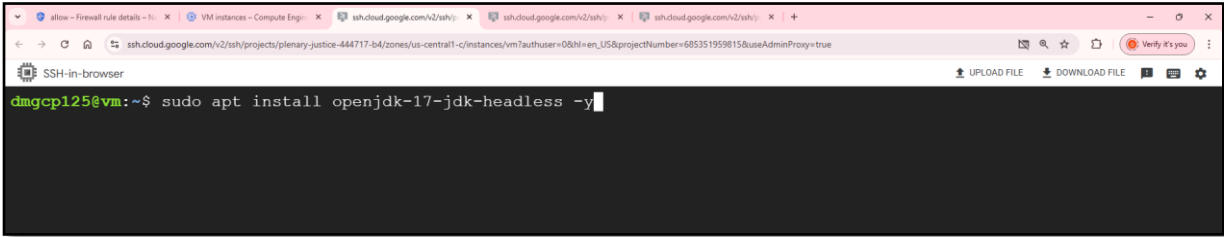
13

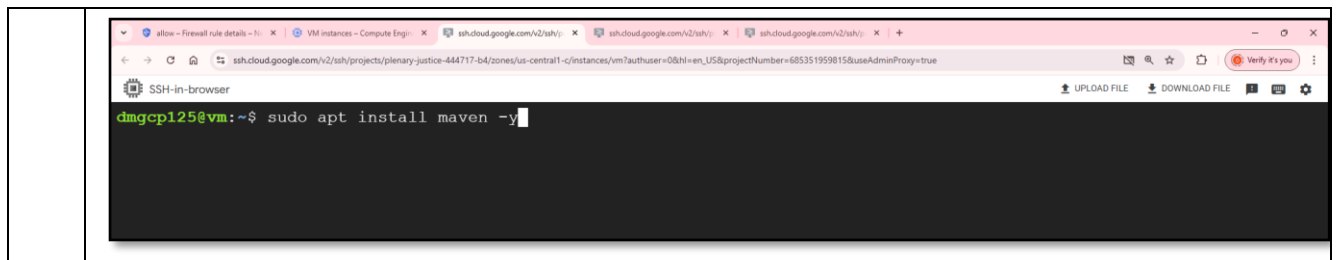
ubuntu packages
sudo apt update

The screenshot shows a terminal window with the command 'sudo apt update' being executed. The output of the command is not visible in the image.

14

Install docker using command: sudo apt install docker.io

	 If prompted press Y
15	Add current user in docker group using command: <code>sudo usermod -aG docker \$USER</code>  Reboot machine using command: <code>sudo reboot</code>  Note: Docker group is already created as part of docker installation. Check using command: <code>cat /etc/group</code>. If docker group is not present create using command: <code>sudo newgrp docker</code>
16	Check docker installation: <code>docker --version</code> 
17	Install jdk17 Command: <code>sudo apt install openjdk-17-jdk-headless -y</code> 
18	Install maven Command: <code>sudo apt install maven -y</code>



Activity 2: Setup kubectl

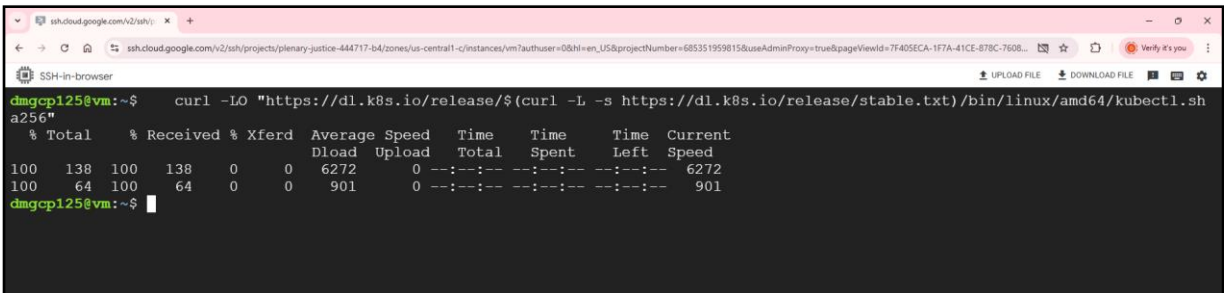
2.1. Prerequisite

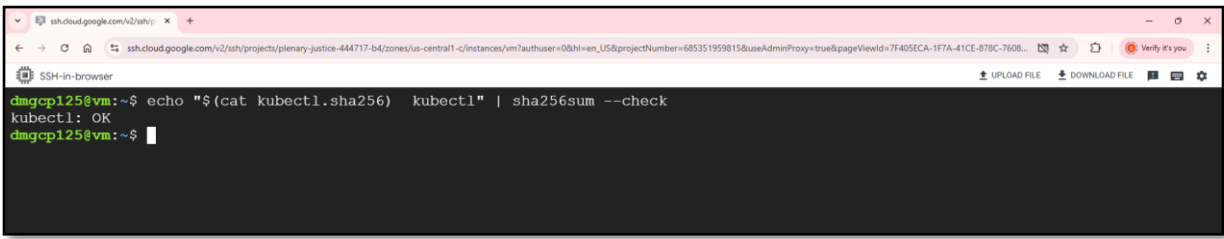
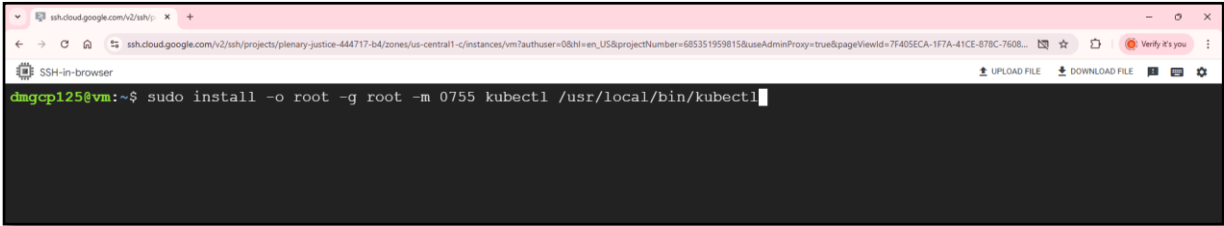
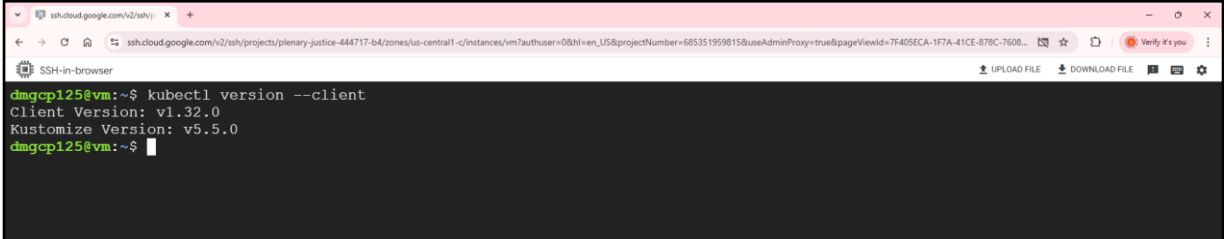
Ubuntu VM with docker installed

2.2. Scenario

Installing kubectl

2.3. Steps

1	Go to vm
2	- Download latest release - Command: <code>curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"</code>  The screenshot shows the command <code>curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"</code> being executed. The output shows the download progress: 100% Total, 138% Received, 100% Xferd, Average Speed 7666, Time 0, Time Spent 0, Time Left 0, Current Speed 118M.
3	- Download kubectl checksum. - Command: <code>curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"</code>  The screenshot shows the command <code>curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"</code> being executed. The output shows the download progress: 100% Total, 138% Received, 100% Xferd, Average Speed 6272, Time 0, Time Spent 0, Time Left 0, Current Speed 901.
4	- Validate kubectl binary. - Command: <code>echo "\$(cat kubectl.sha256) kubectl" sha256sum --check</code>

	
5	• Install kubectl. • Command: <code>sudo install -o root -g root -m 0755 kubect1 /usr/local/bin/kubect1</code> 
6	• Check installation • Command: <code>kubect1 version --client</code> 

Activity 3: Setup minikube cluster

3.1. Prerequisite

Ubuntu VM with docker, kubectl installed

3.2. Scenario

Installing minikube cluster

3.3. Steps

1	Go to vm
2	• Get latest Debian package • Command: <code>curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64</code>

3

- Install minikube
- Command: `sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64`

4

Start minikube

Command: `minikube start --driver=docker --force`

5

Output screen:

6

Execute below command to check installation

Command: `kubectl get po -A`

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-6f6b679f8f-w915h	1/1	Running	1 (81s ago)	4m39s
kube-system	etcd-minikube	1/1	Running	1 (86s ago)	4m44s
kube-system	kube-apiserver-minikube	1/1	Running	1 (76s ago)	4m46s
kube-system	kube-controller-manager-minikube	1/1	Running	1 (86s ago)	4m44s
kube-system	kube-proxy-9s58l	1/1	Running	1 (86s ago)	4m40s
kube-system	kube-scheduler-minikube	1/1	Running	1 (86s ago)	4m44s
kube-system	storage-provisioner	1/1	Running	3 (71s ago)	4m42s

Activity 4: Kubernetes commands

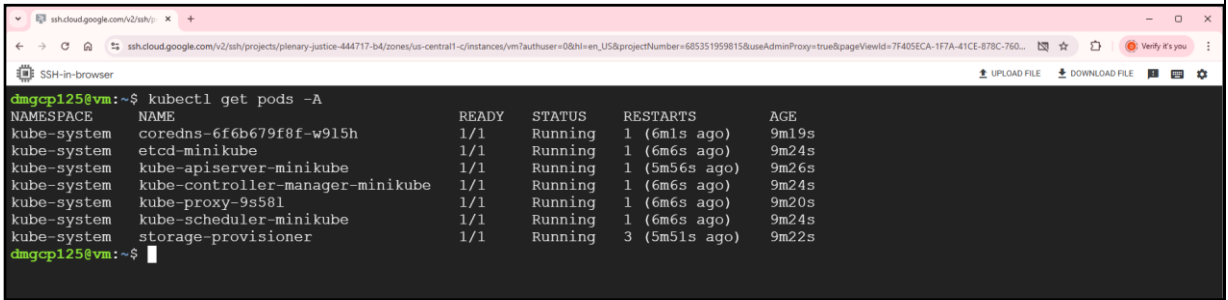
4.1. Prerequisite

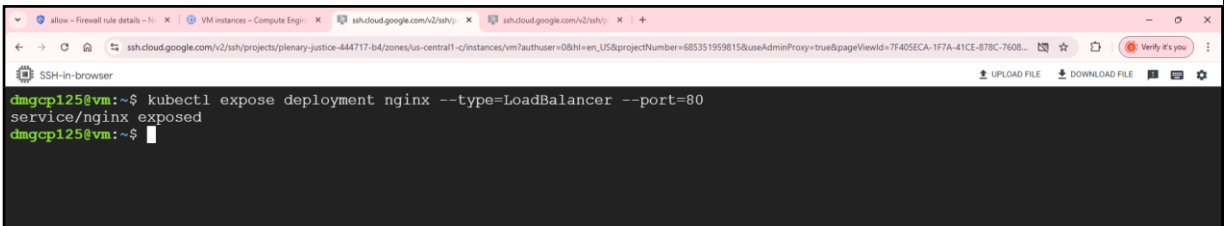
Minikube cluster started

4.2. Scenario

Executing Kubernetes commands

4.3. Steps

1	Go to vm kubectl version --client (check version kubectl) minikube version minikube delete (execute if any issues) minikube start --driver=docker --force																																																
2	Listing pods: Command: kubectl get pods -A or kubectl get po -A  <table><tr><th>NAMESPACE</th><th>NAME</th><th>READY</th><th>STATUS</th><th>RESTARTS</th><th>AGE</th></tr><tr><td>kube-system</td><td>coredns-6f6b679f8f-w915h</td><td>1/1</td><td>Running</td><td>1 (6m1s ago)</td><td>9m19s</td></tr><tr><td>kube-system</td><td>etcd-minikube</td><td>1/1</td><td>Running</td><td>1 (6m6s ago)</td><td>9m24s</td></tr><tr><td>kube-system</td><td>kube-apiserver-minikube</td><td>1/1</td><td>Running</td><td>1 (5m56s ago)</td><td>9m26s</td></tr><tr><td>kube-system</td><td>kube-controller-manager-minikube</td><td>1/1</td><td>Running</td><td>1 (6m6s ago)</td><td>9m24s</td></tr><tr><td>kube-system</td><td>kube-proxy-9s58l</td><td>1/1</td><td>Running</td><td>1 (6m6s ago)</td><td>9m20s</td></tr><tr><td>kube-system</td><td>kube-scheduler-minikube</td><td>1/1</td><td>Running</td><td>1 (6m6s ago)</td><td>9m24s</td></tr><tr><td>kube-system</td><td>storage-provisioner</td><td>1/1</td><td>Running</td><td>3 (5m51s ago)</td><td>9m22s</td></tr></table>	NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	kube-system	coredns-6f6b679f8f-w915h	1/1	Running	1 (6m1s ago)	9m19s	kube-system	etcd-minikube	1/1	Running	1 (6m6s ago)	9m24s	kube-system	kube-apiserver-minikube	1/1	Running	1 (5m56s ago)	9m26s	kube-system	kube-controller-manager-minikube	1/1	Running	1 (6m6s ago)	9m24s	kube-system	kube-proxy-9s58l	1/1	Running	1 (6m6s ago)	9m20s	kube-system	kube-scheduler-minikube	1/1	Running	1 (6m6s ago)	9m24s	kube-system	storage-provisioner	1/1	Running	3 (5m51s ago)	9m22s
NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE																																												
kube-system	coredns-6f6b679f8f-w915h	1/1	Running	1 (6m1s ago)	9m19s																																												
kube-system	etcd-minikube	1/1	Running	1 (6m6s ago)	9m24s																																												
kube-system	kube-apiserver-minikube	1/1	Running	1 (5m56s ago)	9m26s																																												
kube-system	kube-controller-manager-minikube	1/1	Running	1 (6m6s ago)	9m24s																																												
kube-system	kube-proxy-9s58l	1/1	Running	1 (6m6s ago)	9m20s																																												
kube-system	kube-scheduler-minikube	1/1	Running	1 (6m6s ago)	9m24s																																												
kube-system	storage-provisioner	1/1	Running	3 (5m51s ago)	9m22s																																												
3	Create pod using command Command: kubectl run nginx --image=nginx --restart=Never  List pods: Command: kubectl get pods																																																

	 <pre> sshcloud.google.com/v2/ssh/ sshcloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm?authuser=0&hl=en_US&projectNumber=685351959815&useAdminProxy=true&pageViewId=7f405eca-1f7a-41ce-878c-760... SSH-in-browser dmgcp125@vm:~\$ kubectl get pods NAME READY STATUS RESTARTS AGE nginx 1/1 Running 0 60s dmgcp125@vm:~\$ </pre>
4	Delete pod Command: <code>kubectl delete pod nginx</code>  <pre> sshcloud.google.com/v2/ssh/ sshcloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm?authuser=0&hl=en_US&projectNumber=685351959815&useAdminProxy=true&pageViewId=7f405eca-1f7a-41ce-878c-760... SSH-in-browser dmgcp125@vm:~\$ kubectl delete pod nginx pod "nginx" deleted dmgcp125@vm:~\$ </pre>
5	Create deployment Command: <code>kubectl create deployment nginx --image=nginx:latest</code>  <pre> sshcloud.google.com/v2/ssh/ sshcloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm?authuser=0&hl=en_US&projectNumber=685351959815&useAdminProxy=true&pageViewId=7f405eca-1f7a-41ce-878c-760... SSH-in-browser dmgcp125@vm:~\$ kubectl create deployment nginx --image=nginx:latest deployment.apps/nginx created dmgcp125@vm:~\$ </pre> List deployment Command: <code>kubectl get deployment</code>  <pre> sshcloud.google.com/v2/ssh/ sshcloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm?authuser=0&hl=en_US&projectNumber=685351959815&useAdminProxy=true&pageViewId=7f405eca-1f7a-41ce-878c-760... SSH-in-browser dmgcp125@vm:~\$ kubectl get deployment NAME READY UP-TO-DATE AVAILABLE AGE nginx 1/1 1 1 2m35s dmgcp125@vm:~\$ </pre>
6	Expose deployment Command: <code>kubectl expose deployment nginx --type=LoadBalancer --port=80</code>  <pre> allow - Firewall rule details - h VM instances - Compute Engi sshcloud.google.com/v2/ssh/ sshcloud.google.com/v2/ssh/ sshcloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm?authuser=0&hl=en_US&projectNumber=685351959815&useAdminProxy=true&pageViewId=7f405eca-1f7a-41ce-878c-760... SSH-in-browser dmgcp125@vm:~\$ kubectl expose deployment nginx --type=LoadBalancer --port=80 service/nginx exposed dmgcp125@vm:~\$ </pre> List services

Command: kubectl get services

```
dmgcp125@vm:~$ kubectl get svc
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes   ClusterIP   10.96.0.1     <none>         443/TCP    106m
nginx        LoadBalancer 10.106.21.72  192.168.49.2   80:31528/TCP 23s
dmgcp125@vm:~$
```

7

To access load balancer service start minikube tunnel
Command: minikube tunnel

```
dmgcp125@vm:~$ minikube tunnel

Status:
  machine: minikube
  pid: 44811
  route: 10.96.0.0/12 -> 192.168.49.2
  minikube: Running
  services: [nginx]
  errors:
    minikube: no errors
    router: no errors
    loadbalancer emulator: no errors
dmgcp125@vm:~$
```

Open another terminal

.Get ip address :

Command: kubectl get service nginx

```
dmgcp125@vm:~$ kubectl get service nginx
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
nginx        LoadBalancer 10.106.21.72  192.168.49.2   80:31528/TCP 2m21s
dmgcp125@vm:~$
```

Access application with curl

command: curl 192.168.49.2:80 (replace ip with external-ip from previous step)

```
dmgcp125@vm:~$ curl 192.168.49.2:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
dmgcp125@vm:~$
```

8

Check logs of pod

Get pod name using command; kubectl get pods

Use same name in below command
Command: kubectl logs <pod-name>

```
ssh.cloud.google.com/v2/ssh/ x ssh.cloud.google.com/v2/ssh/ x +
ssh.cloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm7authuser=08h=en_US&projectNumber=685351959815&useAdminProxy=true
SSH-in-browser
dmgcp125@vm:~/easyfly$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-c95765fd4-59dcv              1/1     Running   0           25s
dmgcp125@vm:~/easyfly$ kubectl logs nginx-c95765fd4-59dcv
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/01/13 10:24:12 [notice] 1#1: using the "epoll" event method
2025/01/13 10:24:12 [notice] 1#1: nginx/1.27.3
2025/01/13 10:24:12 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
2025/01/13 10:24:12 [notice] 1#1: OS: Linux 5.15.0-1073-gcp
2025/01/13 10:24:12 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/01/13 10:24:12 [notice] 1#1: start worker processes
2025/01/13 10:24:12 [notice] 1#1: start worker process 29
2025/01/13 10:24:12 [notice] 1#1: start worker process 30
2025/01/13 10:24:12 [notice] 1#1: start worker process 31
2025/01/13 10:24:12 [notice] 1#1: start worker process 32
2025/01/13 10:24:12 [notice] 1#1: start worker process 33
2025/01/13 10:24:12 [notice] 1#1: start worker process 34
2025/01/13 10:24:12 [notice] 1#1: start worker process 35
2025/01/13 10:24:12 [notice] 1#1: start worker process 36
dmgcp125@vm:~/easyfly$
```

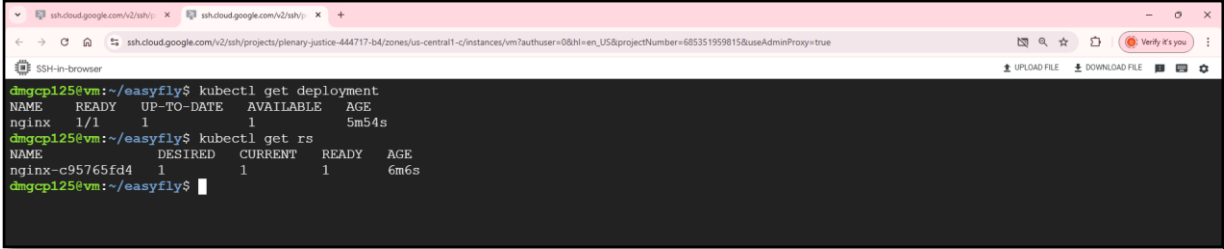
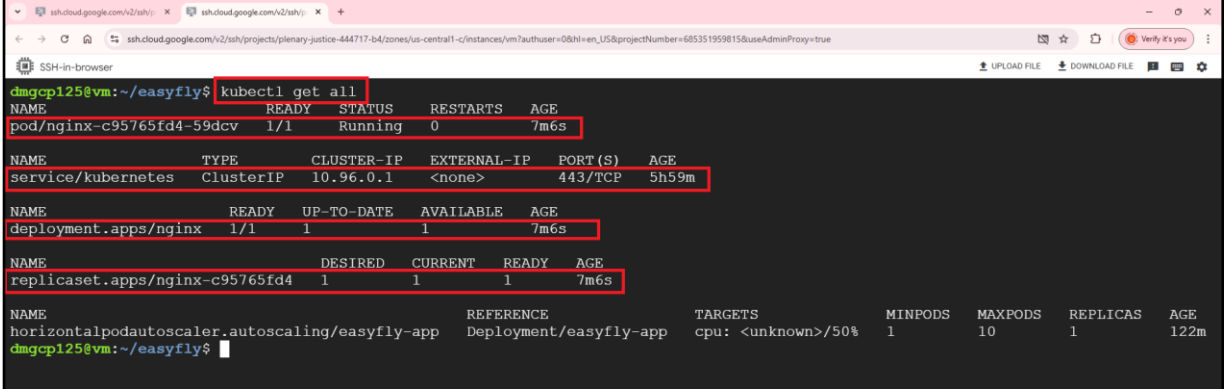
9 Get pod in-depth details
Command: kubectl get pods -o wide

```
ssh.cloud.google.com/v2/ssh/ x ssh.cloud.google.com/v2/ssh/ x +
ssh.cloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm7authuser=08h=en_US&projectNumber=685351959815&useAdminProxy=true
SSH-in-browser
dmgcp125@vm:~/easyfly$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
nginx-c95765fd4-59dcv              1/1     Running   0           2m43s  10.244.0.63   minikube   <none>            <none>
dmgcp125@vm:~/easyfly$
```

10 Explain pod
Command: kubectl describe pod <pod-name>

```
ssh.cloud.google.com/v2/ssh/ x ssh.cloud.google.com/v2/ssh/ x +
ssh.cloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm7authuser=08h=en_US&projectNumber=685351959815&useAdminProxy=true
SSH-in-browser
dmgcp125@vm:~/easyfly$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-c95765fd4-59dcv              1/1     Running   0           3m37s
dmgcp125@vm:~/easyfly$ kubectl describe pod nginx-c95765fd4-59dcv
Name:                               nginx-c95765fd4-59dcv
Namespace:                         default
Priority:                           0
Service Account:                   default
Node:                               minikube/192.168.49.2
Start Time:                        Mon, 13 Jan 2025 10:24:11 +0000
Labels:                             app=nginx
                                     pod-template-hash=c95765fd4
Annotations:                        <none>
Status:                             Running
IP:                                 10.244.0.63
IPs:                                10.244.0.63
Controlled By:                      ReplicaSet/nginx-c95765fd4
Containers:
  nginx:
    Container ID:   docker://b8314309b4b41bdbc77f4b0adac77ebcc7bdbc14868b737cf46feaa875298c1f4
    Image:          nginx:latest
    Image ID:       docker-pullable://nginx@sha256:42e917aaa1b5bb40dd0f6f7f4f857490ac7747d7ef73b391c774a41a8b994f15
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Mon, 13 Jan 2025 10:24:12 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-xqvn1 (ro)
Conditions:
  Type                               Status
PodReadyToStartContainers            True
```

11 Get replicaset
Command: kubectl get rs

	 <pre> dmgcp125@vm:~/easyfly\$ kubectl get deployment NAME READY UP-TO-DATE AVAILABLE AGE nginx 1/1 1 1 5m54s dmgcp125@vm:~/easyfly\$ kubectl get rs NAME DESIRED CURRENT READY AGE nginx-c95765fd4 1 1 1 6m6s dmgcp125@vm:~/easyfly\$ </pre>
12	Get all resources Command: kubectl get all minikube dashboard  <pre> dmgcp125@vm:~/easyfly\$ kubectl get all NAME READY STATUS RESTARTS AGE pod/nginx-c95765fd4-59dcv 1/1 Running 0 7m6s NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE service/kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 5h59m NAME READY UP-TO-DATE AVAILABLE AGE deployment.apps/nginx 1/1 1 1 7m6s NAME DESIRED CURRENT READY AGE replicaset.apps/nginx-c95765fd4 1 1 1 7m6s NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE horizontalpodautoscaler.autoscaling/easyfly-app Deployment/easyfly-app cpu: <unknown>/50% 1 10 1 122m dmgcp125@vm:~/easyfly\$ </pre>
13	Get deployment in yaml format Command: kubectl get deployment nginx -o yaml >> deployment.yaml cat deployment.yaml <pre> apiVersion: apps/v1 kind: Deployment metadata: annotations: deployment.kubernetes.io/revision: "1" creationTimestamp: "2025-01-13T10:24:11Z" generation: 1 labels: app: nginx name: nginx namespace: default resourceVersion: "40096" uid: 97b2cf77-f6ec-4645-a0d8-5405c4ea2d85 spec: progressDeadlineSeconds: 600 replicas: 1 revisionHistoryLimit: 10 selector: </pre>
14	Create pod using yaml file vi pod.yaml

	after Entering below content first press the Esc key to enter command mode :wq (to save and quit), and press Enter. <pre>apiVersion: v1 kind: Pod metadata: name: nginx-pod-using-yaml-file labels: app: nginx spec: containers: - name: nginx image: nginx ports: - containerPort: 80</pre> Create pod Command: <code>kubectl apply -f pod.yml</code>
--	--

Activity 5: Deploy Spring Boot Application

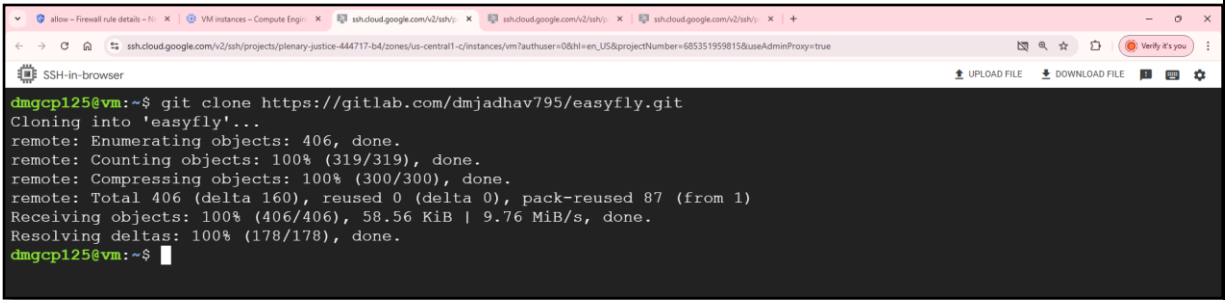
5.1. Prerequisite

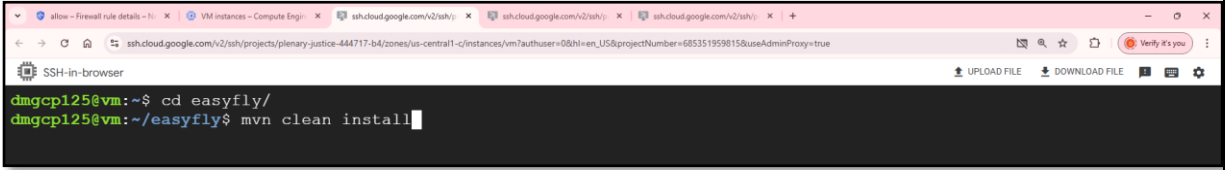
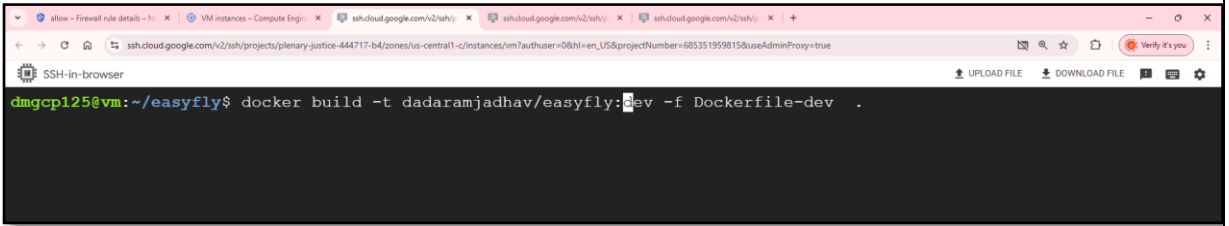
Minikube cluster started.

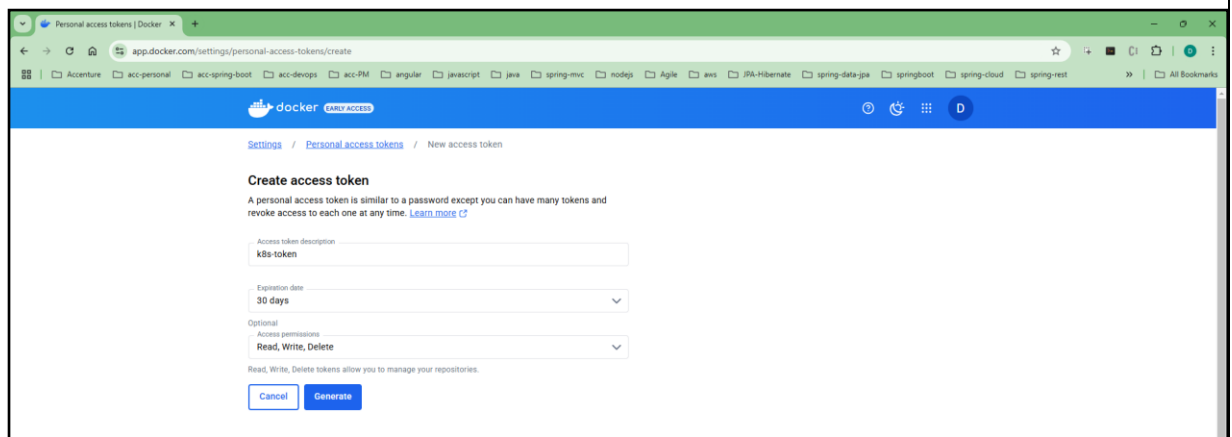
5.2. Scenario

Deploying spring boot application in minikube cluster

5.3. Steps

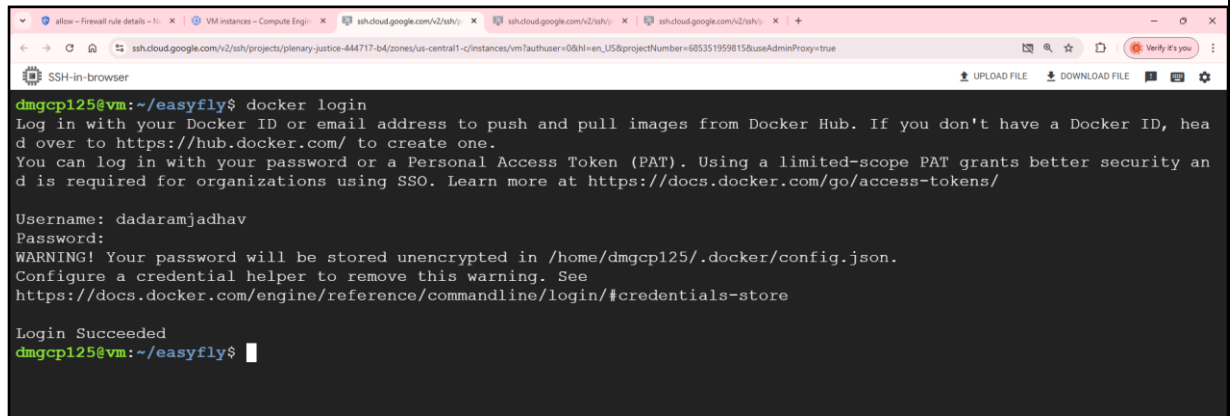
1	Go to vm Login to GitLab account Fork Repository https://gitlab.com/dadaramjadhav795/easyfly.git
2	Remove easyfly application if present <code>rm -r easyfly</code> clone the repository git clone https://gitlab.com/dadaramjadhav795/easyfly.git 
3	Build application Command: <code>cd easyfly</code>

	mvn clean install Is target 
4	Create dockerfile in root directory. <pre>touch Dockerfile-dev vim Dockerfile-dev</pre> To insert text, press `i`. After typing your content, press `Esc` to exit insert mode. If you want to save and exit at the same time, type `:wq` and press `Enter`. <pre>cat Dockerfile-dev</pre> <pre>#this is dockerfile for development #use Dockerfile-dev and edit it FROM --platform=linux/arm64 eclipse-temurin:17-jdk COPY target/*.jar app.jar ENV server.port=8081 ENTRYPOINT ["java", "-Dspring.profiles.active=dev", "-jar", "/app.jar"]</pre>
5	Login to Docker Hub docker login enter username of Docker Hub account and enter pass code Build docker image Command: <code>docker build -t dadaramjadhav/easyfly:dev -f Dockerfile-dev</code> 
6	Create personal access token in dockerhub - Login into docker hub - Go to https://app.docker.com/settings/personal-access-tokens - Click on generate new token - Enter token name - Choose expiration time - Choose access permission (Read, Write, Delete) - Click on generate

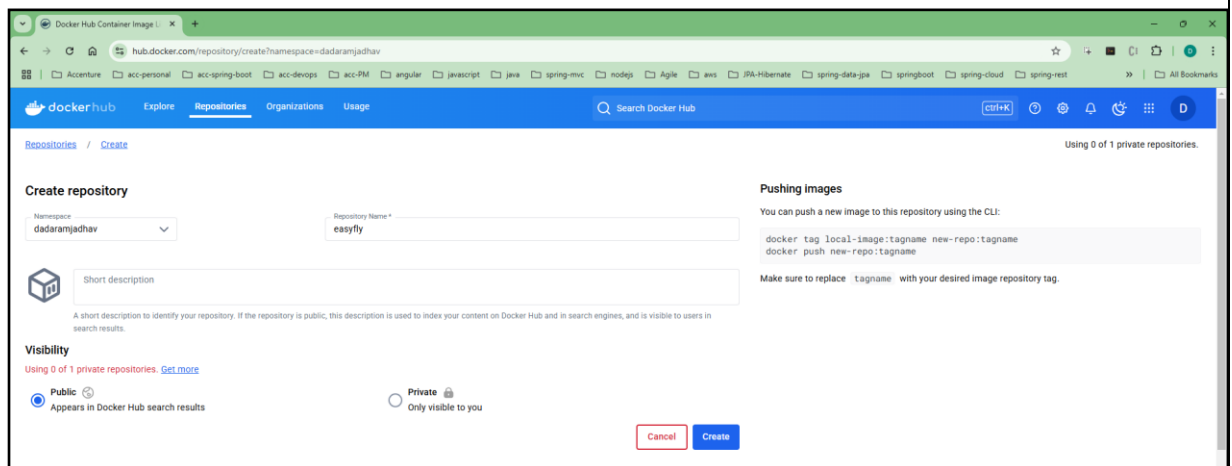


h. Copy personal access token generated

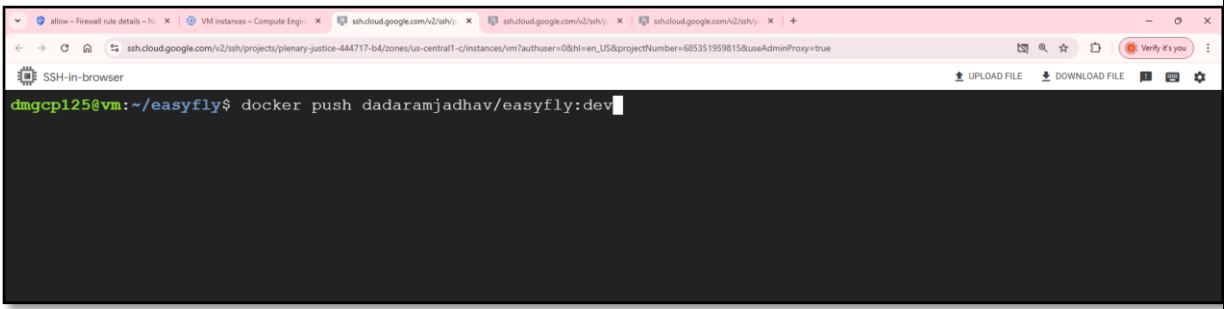
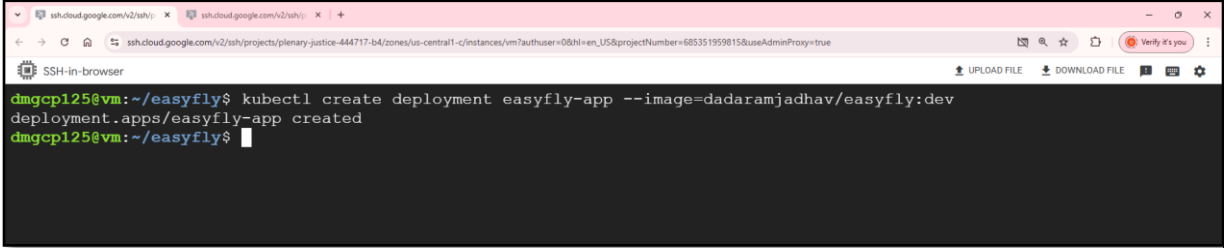
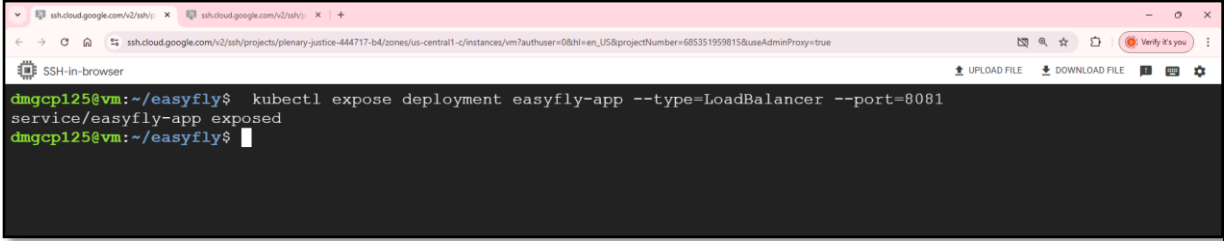
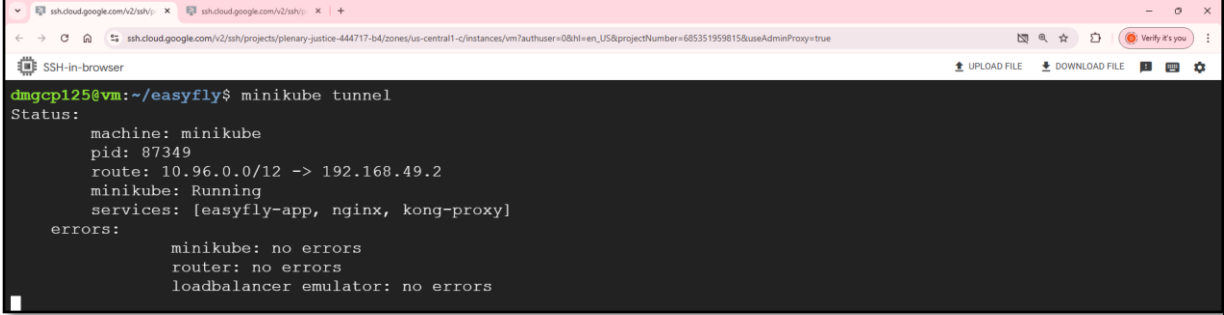
- 7 Login into docker using command docker login
 Username: Enter dockerhub username
 Password: enter personal access token (created in previous step)

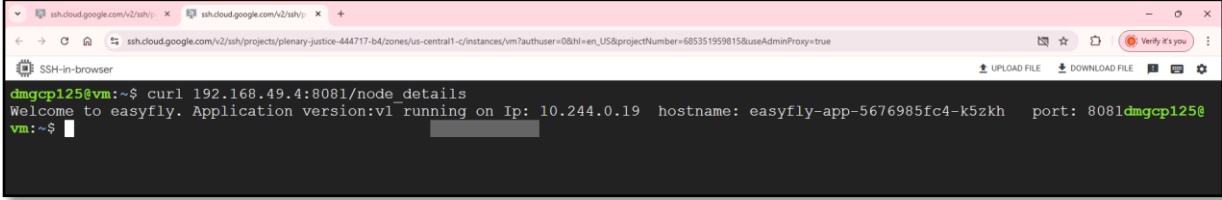


- 8 Create registry into docker hub
- Go to <https://hub.docker.com/>
 - Click on create repository
 - Enter repository name and click on create



- 9 Push image to docker hub registry
 Check list of images: docker images
 Command: docker push dadaramjadhav795/easyfly:dev

	
10	Create deployment Command: <code>kubectl create deployment easyfly-app --image= dadaramjadhav/easyfly:dev</code> 
11	Expose deployment Command: <code>kubectl expose deployment easyfly-app --type=LoadBalancer --port=8081</code> check deployments <code>kubectl get deployments</code> check pods <code>kubectl get pods</code> check log of created pod <code>kubect logs <pod-id></code> 
12	Start minikube tunnel to access loadbalancer service using command <code>minikube tunnel</code>  Keep this terminal running
13	Open another terminal List services

	Command: <code>kubectl get svc</code> 
14	Access application Command: <code>curl 192.168.49.4:8081/node_details</code>  Other endpoints <code>curl 192.168.49.4:8081/api-v1/flights/ (GET)</code> <code>curl 192.168.49.4:8081/api-v1/flights/3 (GET)</code> <code>curl --header "Content-Type: application/json" --request POST --data '{"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/ (POST Request)</code> <code>curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6,"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/6 (PUT request)</code> <code>curl --request DELETE 192.168.49.4:8081/api-v1/flights/6 (DELETE)</code>

Activity 6: Scale spring boot application

6.1. Prerequisite

Spring boot application running inside minikube cluster

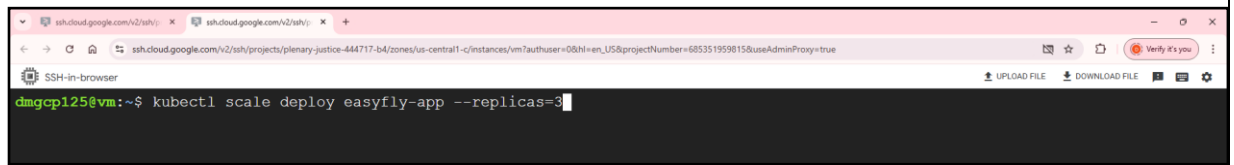
6.2. Scenario

Scaling spring boot application to handle increasing load

6.3. Steps

1	Go to vm
2	Scale deployment

Command: `kubectl scale deploy easyfly-app --replicas=3`



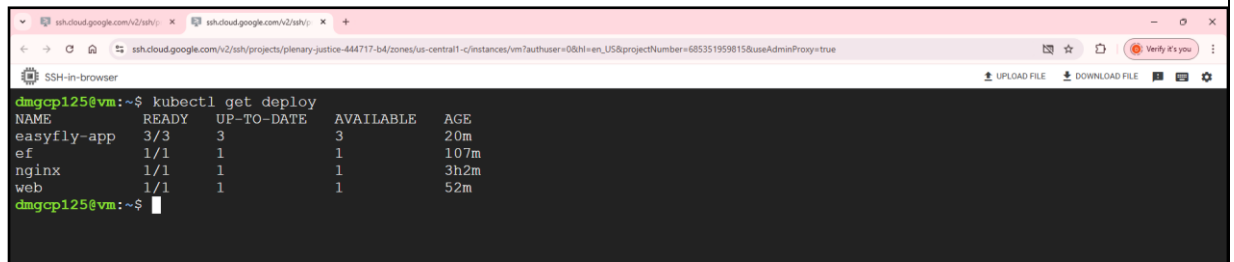
```
dmgcp125@vm:~$ kubectl scale deploy easyfly-app --replicas=3
```

Check deployment

Command: `kubectl get deploy`

check pods

`kubectl get pods`



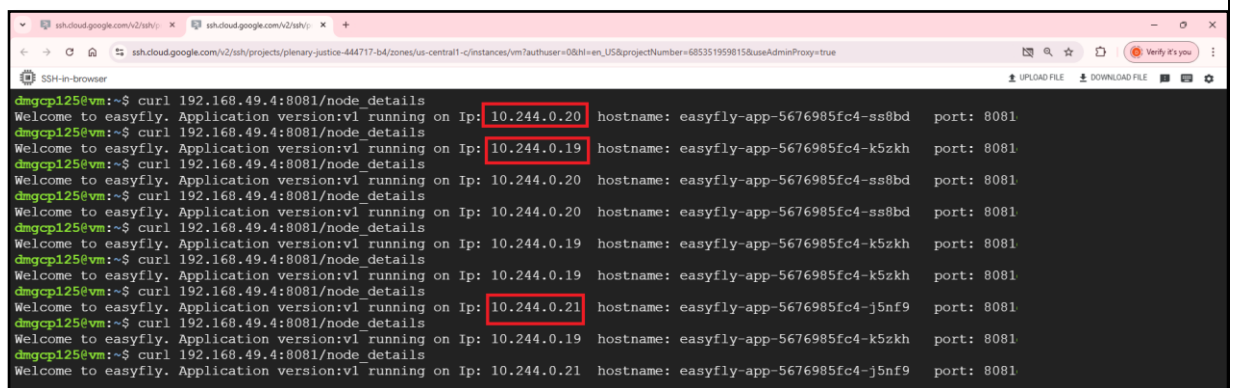
```
dmgcp125@vm:~$ kubectl get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
easyfly-app   3/3     3             3            20m
ef            1/1     1             1           107m
nginx         1/1     1             1           3h2m
web           1/1     1             1            52m
dmgcp125@vm:~$
```

3

Access application

Run application for 5-6 times. You will get response from different container.

Command: `curl 192.168.49.4:8081/node_details`



```
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.20 hostname: easyfly-app-5676985fc4-ss8bd port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.19 hostname: easyfly-app-5676985fc4-k5zkh port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.20 hostname: easyfly-app-5676985fc4-ss8bd port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.20 hostname: easyfly-app-5676985fc4-ss8bd port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.19 hostname: easyfly-app-5676985fc4-k5zkh port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.19 hostname: easyfly-app-5676985fc4-k5zkh port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.19 hostname: easyfly-app-5676985fc4-k5zkh port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.21 hostname: easyfly-app-5676985fc4-j5nf9 port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.19 hostname: easyfly-app-5676985fc4-k5zkh port: 8081
dmgcp125@vm:~$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.21 hostname: easyfly-app-5676985fc4-j5nf9 port: 8081
```

4

Scale down application

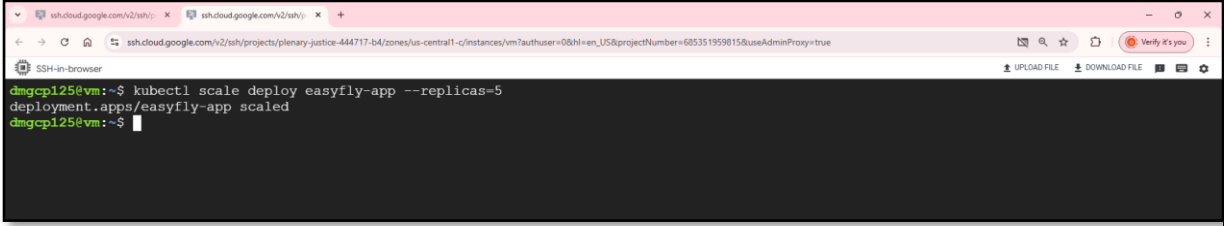
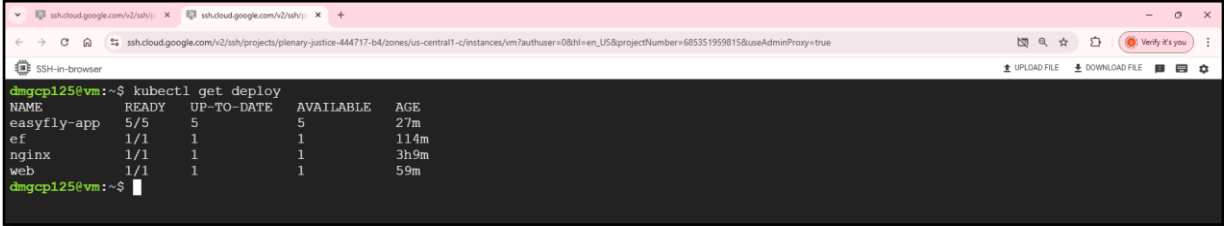
Command: `kubectl scale deploy easyfly-app --replicas=2`



```
dmgcp125@vm:~$ kubectl scale deploy easyfly-app --replicas=2
deployment.apps/easyfly-app scaled
dmgcp125@vm:~$
```

Get deployment

Command: `kubectl get deploy`

	 <pre> dmgcp125@vm:~\$ kubectl get deploy NAME READY UP-TO-DATE AVAILABLE AGE easyfly-app 2/2 2 2 26m ef 1/1 1 1 113m nginx 1/1 1 1 3h8m web 1/1 1 1 58m dmgcp125@vm:~\$ </pre> Keep hitting this endpoint to confirm scaling curl 192.168.49.4:8081/node_details
5	Scale up application Command: kubectl scale deploy easyfly-app --replicas=5  <pre> dmgcp125@vm:~\$ kubectl scale deploy easyfly-app --replicas=5 deployment.apps/easyfly-app scaled dmgcp125@vm:~\$ </pre> List deployment Command: kubectl get deploy  <pre> dmgcp125@vm:~\$ kubectl get deploy NAME READY UP-TO-DATE AVAILABLE AGE easyfly-app 5/5 5 5 27m ef 1/1 1 1 114m nginx 1/1 1 1 3h9m web 1/1 1 1 59m dmgcp125@vm:~\$ </pre> Keep hitting this endpoint to confirm scaling curl 192.168.49.4:8081/node_details

Activity 7: Autoscaling spring boot application

7.1. Prerequisite

Spring boot application running

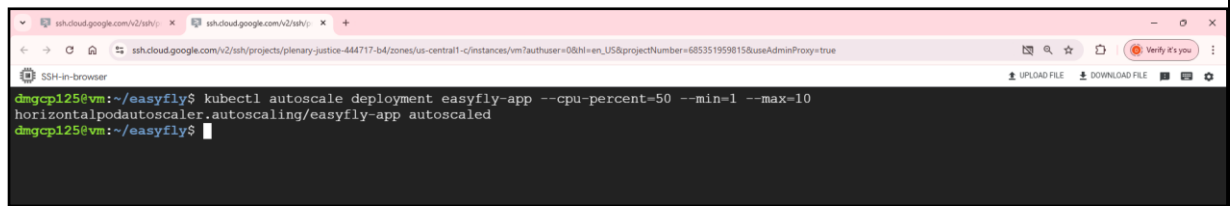
7.2. Scenario

Autoscaling spring boot application to handle dynamic load

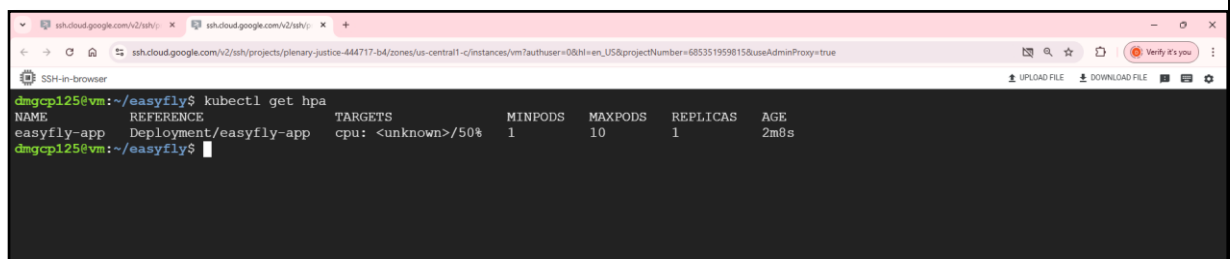
7.3. Steps

1	Go to vm
2.	Autoscale application based on CPU utilization

Command: `kubectl autoscale deployment easyfly-app --cpu-percent=50 --min=1 --max=10`



- This defines a Horizontal Autoscaler that maintains between 1 and 10 replicas of the Pods controlled by the easyfly-app deployment.
- It will increase and decrease the number of replicas (via the deployment) so as to maintain an average CPU utilization across all Pods to 50%.
- Check status of auto scalar with
- Command: `kubectl get hpa`



Activity 8: Deploying Spring Boot Application using manifest file

8.1. Prerequisite

Spring boot application

8.2. Scenario

Deploy spring boot application using manifest files

8.3. Steps

1	Go to vm
2.	Remove easyfly directory if present Rm -r easyfly Clone source code Command: git clone https://gitlab.com/dadaramjadhav795/easyfly.git Change directory cd easyfly
3	Build application Command: mvn clean install

	
4	touch Dockerfile-dev vim Dockerfile-dev To insert text, press `i`. After typing your content, press `Esc` to exit insert mode. If you want to save and exit at the same time, type `:wq` and press `Enter`. cat Dockerfile-dev <pre>#this is dockerfile for development #use Dockerfile-dev and edit it FROM --platform=linux/arm64 eclipse-temurin:17-jdk COPY target/*.jar app.jar ENV server.port=8081 ENTRYPOINT ["java", "-Dspring.profiles.active=dev", "-jar", "/app.jar"]</pre> Build docker image Command: docker build -t dadaramjadhav/easyfly -f Dockerfile-dev
5	Push docker image to docker hub Command: docker push dadaramjadhav/easyfly:latest 
6	Deploy application using below manifest file Use command: vim deployment.yml To insert text, press `i`. After typing your content, press `Esc` to exit insert mode. If you want to save and exit at the same time, type `:wq` and press `Enter`. cat deployment.yml <pre>apiVersion: apps/v1 kind: Deployment metadata: name: easyfly spec: selector: matchLabels: app: easyfly replicas: 3 template:</pre>

```
metadata:
  labels:
    app: easyfly
spec:
  containers:
    - name: easyfly
      image: dadaramjadhav/easyfly:dev
      ports:
        - containerPort: 8081
---

apiVersion: v1
kind: Service
metadata:
  name: easyfly-svc
spec:
  selector:
    app: easyfly
  ports:
    - protocol: "TCP"
      port: 8081
      targetPort:
  type: LoadBalancer
```

Deployment
Command: `kubectl apply -f deployment.yml`



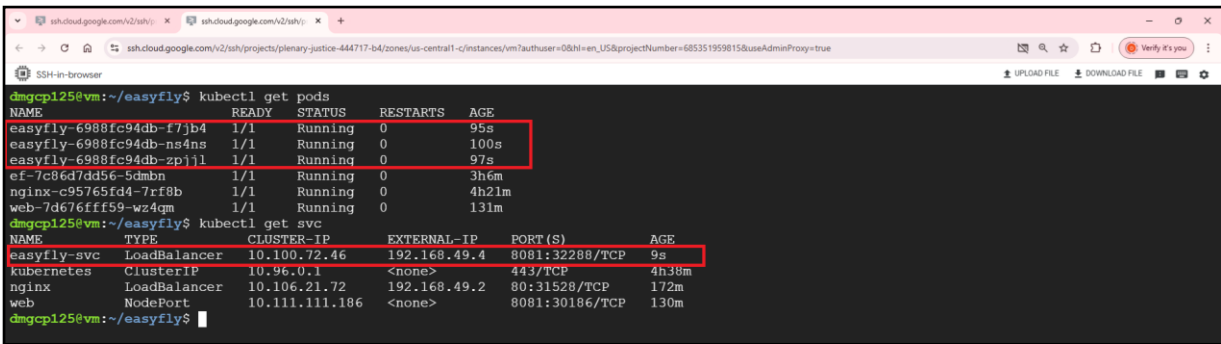
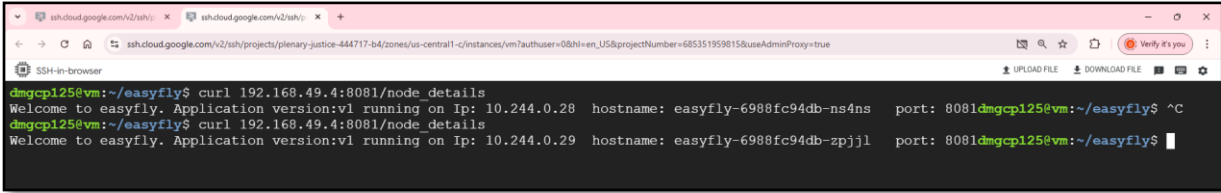
check deployment
`kubectl get deployments`

check pods
`kubectl get pods`

check services
`kubectl get svc`

start minikube tunnel
`minikube tunnel`

open new terminal

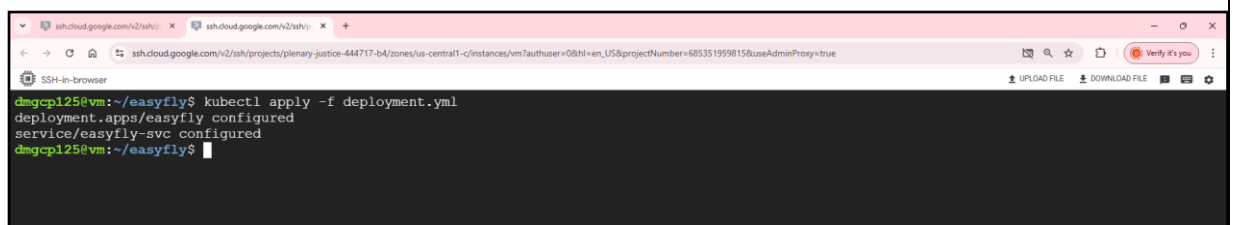
	 <pre> dmgcp125@vm:~/easyfly\$ kubectl get pods NAME READY STATUS RESTARTS AGE easyfly-6988fc94db-f7jb4 1/1 Running 0 95s easyfly-6988fc94db-ns4ns 1/1 Running 0 100s easyfly-6988fc94db-zpj1l 1/1 Running 0 97s ef-7c86d7dd56-5dmbn 1/1 Running 0 3h6m nginx-c95765fd4-7rf8b 1/1 Running 0 4h21m web-7d676fff59-wz4qm 1/1 Running 0 131m dmgcp125@vm:~/easyfly\$ kubectl get svc NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE easyfly-svc LoadBalancer 10.100.72.46 192.168.49.4 8081:32288/TCP 9s kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 4h39m nginx LoadBalancer 10.106.21.72 192.168.49.2 80:31528/TCP 172m web NodePort 10.111.111.186 <none> 8081:30186/TCP 130m </pre>
7	Access application using loadbalacer ip Command: curl 192.168.49.4:8081/node_details  <pre> dmgcp125@vm:~/easyfly\$ curl 192.168.49.4:8081/node_details Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.28 hostname: easyfly-6988fc94db-ns4ns port: 8081 dmgcp125@vm:~/easyfly\$ curl 192.168.49.4:8081/node_details Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.29 hostname: easyfly-6988fc94db-zpj1l port: 8081 </pre> Try Other endpoints 1. curl 192.168.49.4:8081/api-v1/flights/ (GET) 2. curl 192.168.49.4:8081/api-v1/flights/3 (GET) 3. curl --header "Content-Type: application/json" --request POST --data '{"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/ (POST Request) 4. curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6,"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/6 (PUT request) 5. curl --request DELETE 192.168.49.4:8081/api-v1/flights/6 (DELETE)
8	Scaling application Update deployment.yml (update replica to 5) vim deployment.yml cat deployment.yml <pre> apiVersion: apps/v1 kind: Deployment metadata: name: easyfly spec: selector: matchLabels: app: easyfly replicas: 5 template: </pre>

```
metadata:
  labels:
    app: easyfly
spec:
  containers:
    - name: easyfly
      image: deepakmoud/easyfly:dev
  ports:
    - containerPort: 8081
```

```
apiVersion: v1
kind: Service
metadata:
  name: easyfly-svc
spec:
  selector:
    app: easyfly
  ports:
    - protocol: "TCP"
      port: 8081
      targetPort:
  type: LoadBalancer
```

Apply deployment

Command: `kubectl apply -f deployment.yml`



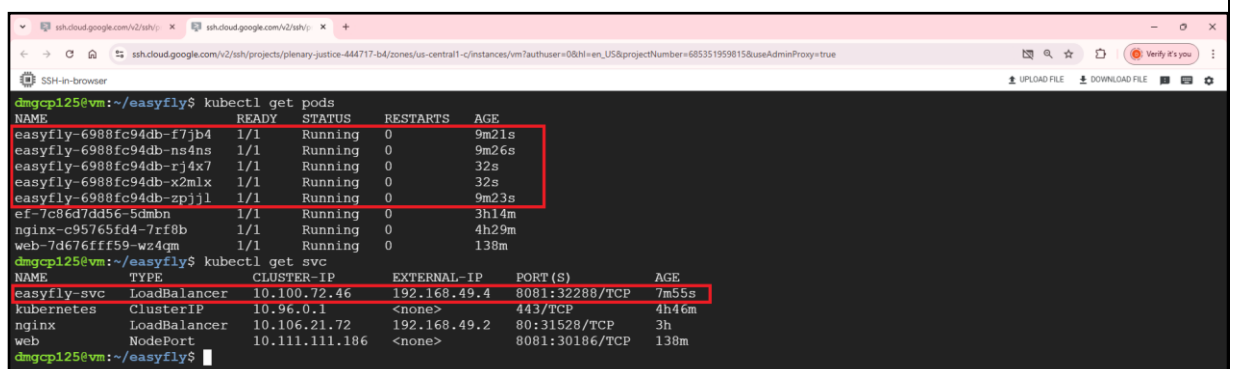
```
dmgcp125@vm:~/easyfly$ kubectl apply -f deployment.yml
deployment.apps/easyfly configured
service/easyfly-svc configured
dmgcp125@vm:~/easyfly$
```

Check deployment

Command:

`kubectl get pods`

`kubectl get svc`



```
dmgcp125@vm:~/easyfly$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
easyfly-6988fc94db-f7jb4            1/1     Running   0           9m21s
easyfly-6988fc94db-ns4ns            1/1     Running   0           9m26s
easyfly-6988fc94db-rj4x7            1/1     Running   0           32s
easyfly-6988fc94db-x2mlx            1/1     Running   0           32s
easyfly-6988fc94db-zpj1l            1/1     Running   0           9m23s
ef-7c86d7d356-5dmbn                 1/1     Running   0           3h14m
nginx-c95765fd4-7rf8b               1/1     Running   0           4h29m
web-7d676fff59-wz4qm                1/1     Running   0           130m
dmgcp125@vm:~/easyfly$ kubectl get svc
NAME      TYPE           CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
easyfly-svc  LoadBalancer  10.100.72.46  192.168.49.4  8081:32288/TCP   7m55s
kubernetes  ClusterIP      10.96.0.1    <none>        443/TCP          4h46m
nginx      LoadBalancer  10.106.21.72  192.168.49.2  80:31528/TCP     3h
web        NodePort       10.111.111.186 <none>        8081:30186/TCP   130m
dmgcp125@vm:~/easyfly$
```

9

Access scaled application

Command: curl 192.168.49.4:8081/node_details

```

dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.31 hostname: easyfly-6988fc94db-x2mlx port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.29 hostname: easyfly-6988fc94db-zpjjl port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.31 hostname: easyfly-6988fc94db-x2mlx port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.29 hostname: easyfly-6988fc94db-zpjjl port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.30 hostname: easyfly-6988fc94db-f7jb4 port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.31 hostname: easyfly-6988fc94db-x2mlx port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.31 hostname: easyfly-6988fc94db-x2mlx port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.28 hostname: easyfly-6988fc94db-ns4ns port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.28 hostname: easyfly-6988fc94db-ns4ns port: 8081dmgcp125@vm:~/easyfly$ ^C
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v1 running on Ip: 10.244.0.32 hostname: easyfly-6988fc94db-rj4x7 port: 8081dmgcp125@vm:~/easyfly$

```

Try Other endpoints

1. curl 192.168.49.4:8081/api-v1/flights/ (GET)
2. curl 192.168.49.4:8081/api-v1/flights/3 (GET)
3. curl --header "Content-Type: application/json" --request POST --data '{"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/ (POST Request)
4. curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6,"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/6 (PUT request)
5. curl --request DELETE 192.168.49.4:8081/api-v1/flights/6 (DELETE)

Activity 9: Changing Deployment Image

9.1. Prerequisite

Spring boot application

9.2. Scenario

Changing deployment image for spring boot application using manifest files

9.3. Steps

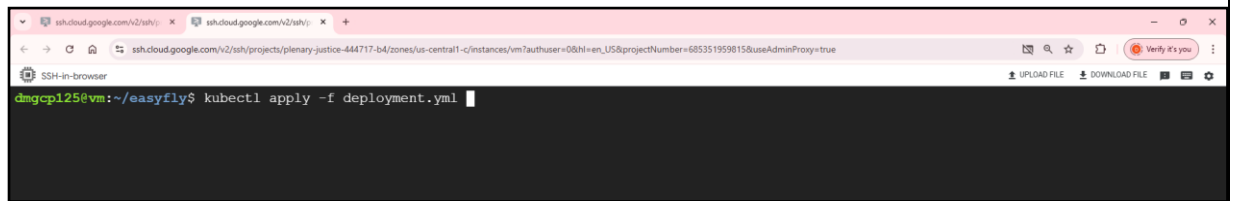
1	Go to vm
2	Update source code to api-v2
3	Pull updated source code Command: git pull
4	Build application

	
5	Build docker image Command: <code>docker build -t dadaramjadhav/easyfly:dev-v2 -f Dockerfile-dev .</code> 
6	Push image into docker hub Command: <code>docker push dadaramjadhav/easyfly:dev-v2</code> 
7	Updated deployment.yml <pre> apiVersion: apps/v1 kind: Deployment metadata: name: easyfly spec: selector: matchLabels: app: easyfly replicas: 5 template: metadata: labels: app: easyfly spec: containers: - name: easyfly image: dadaramjadhav/easyfly:dev-v2 ports: - containerPort: 8081 ---</pre> <pre> apiVersion: v1 kind: Service metadata: name: easyfly-svc </pre>

```
spec:
  selector:
    app: easyfly
  ports:
    - protocol: "TCP"
      port: 8081
      targetPort:
type: LoadBalancer
```

Update deployment

Command: `kubectl apply -f deployment.yml`



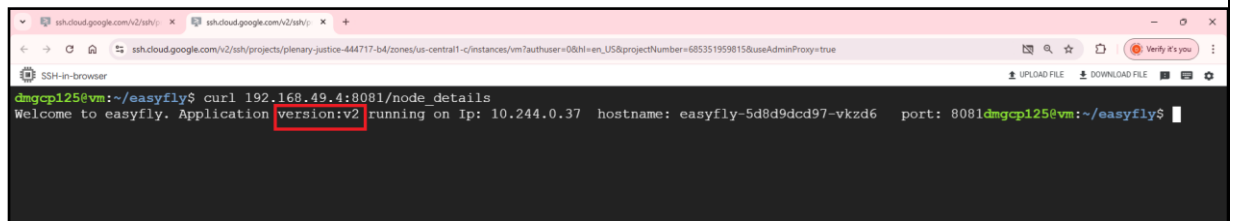
```
dmgcp125@vm:~/easyfly$ kubectl apply -f deployment.yml
```

8

Access application

Access scaled application

Command: `curl 192.168.49.4:8081/node_details`



```
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v2 running on Ip: 10.244.0.37 hostname: easyfly-5d8d9dcd97-vkzd6 port: 8081dmgcp125@vm:~/easyfly$
```

Try Other endpoints

1. `curl 192.168.49.4:8081/api-v2/flights/ (GET)`
2. `curl 192.168.49.4:8081/api-v2/flights/3 (GET)`
3. `curl --header "Content-Type: application/json" --request POST --data '{"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v2/flights/ (POST Request)`
4. `curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6,"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v2/flights/6 (PUT request)`
5. `curl --request DELETE 192.168.49.4:8081/api-v2/flights/6 (DELETE)`

Activity 10: Deploying Spring Boot App with MySQL

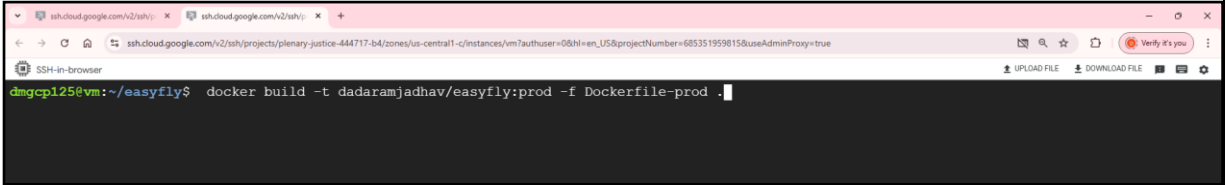
10.1. Prerequisite

Spring boot application with MySQL

10.2. Scenario

Deploying spring boot application with MySQL using manifest files

10.3. Steps

1	Go to vm
2	Build application Command: mvn clean install 
3	Build docker image Command: docker build -t dadaramjadhav/easyfly:prod -f Dockerfile-prod .  Push docker image Command: docker push dadaramjadhav/easyfly:prod 
4	Add db-deployment.yml file. It will create docker container for MySQL <pre>apiVersion: v1 kind: PersistentVolumeClaim metadata: name: mysql-pv-claim labels: app: mysql tier: database spec:</pre>

```
accessModes:
  - ReadWriteOnce
resources:
  requests:
    storage: 1Gi
---

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  labels:
    app: mysql
    tier: database
spec:
  selector:
    matchLabels:
      app: mysql
      tier: database
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: mysql
        tier: database
    spec:
      containers:
        - image: mysql:5.7
          args:
            - "--ignore-db-dir=lost+found"
          name: mysql
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: root
            - name: MYSQL_DATABASE
              value: easyfly_db

      ports:
        - containerPort: 3306
          name: mysql
      volumeMounts:
        - name: mysql-persistent-storage
          mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
```

	<pre> claimName: mysql-pv-claim --- apiVersion: v1 kind: Service metadata: name: mysql labels: app: mysql tier: database spec: ports: - port: 3306 targetPort: 3306 selector: app: mysql tier: database clusterIP: None </pre>
--	--

5

Create MySQL deployment

Command: `kubectl apply -f db-deployment.yml`

```

dmgcp125@vm:~/easyfly$ kubectl apply -f db-deployment.yml

```

Check deployment

Command: `kubectl get pods`

```

dmgcp125@vm:~/easyfly$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
easyfly-5d8d9dcd97-4nvmc            1/1     Running   0           28m
easyfly-5d8d9dcd97-4qlr6            1/1     Running   0           28m
easyfly-5d8d9dcd97-bsrwf            1/1     Running   0           28m
easyfly-5d8d9dcd97-pv5h6            1/1     Running   0           28m
easyfly-5d8d9dcd97-vkzd6            1/1     Running   0           28m
ef-7c86d7dd56-5dmbn                1/1     Running   0           3h57m
mysql-5689f6495f-bd6rm              1/1     Running   0           68s
nginx-c95765fd4-7rf8b               1/1     Running   0           5h12m
web-7d676fff59-wz4qm                1/1     Running   0           3h1m
dmgcp125@vm:~/easyfly$

```

Check database created inside MySQL

Command: `kubectl exec <containername> -it -- /bin/sh`

sh-4.2# `mysql -u root -p`

Enter password:root

Show databases;

Enter exit to come out

```
dmqcp125@vm:~/easyfly$ kubectl exec mysql-5689f6495f-bd6rm -it -- /bin/sh
sh-4.2# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.7.44 MySQL Community Server (GPL)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| easyfly_db |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql>
```

- 6 Deploy spring boot application
deployment.yml is as follows

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: easyfly
spec:
  selector:
    matchLabels:
      app: easyfly
  replicas: 2
  template:
    metadata:
      labels:
        app: easyfly
    spec:
      containers:
        - name: easyfly
          image: dadaramjadhav/easyfly:prod
          ports:
            - containerPort: 8081
```

```
apiVersion: v1
kind: Service
metadata:
  name: easyfly-svc
spec:
  selector:
    app: easyfly
  ports:
    - protocol: "TCP"
      port: 8081
      targetPort:
  type: LoadBalancer
```

Deployment

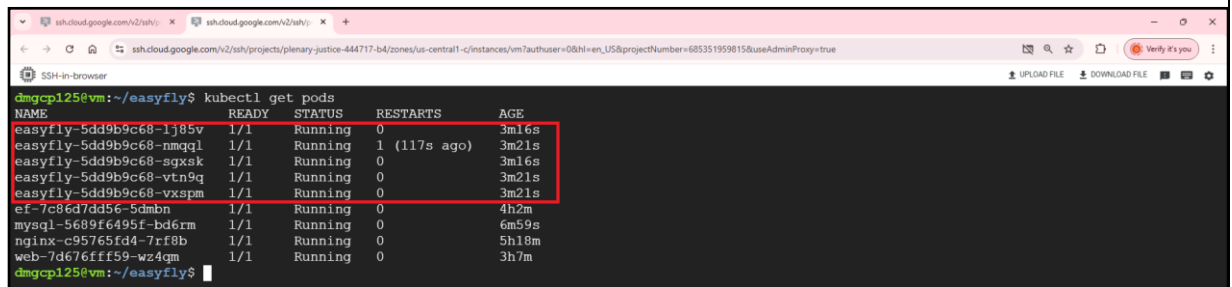
Command: `kubectl apply -f deployment.yml`



```
dmgcp125@vm:~/easyfly$ kubectl apply -f deployment.yml
```

Get list of pods

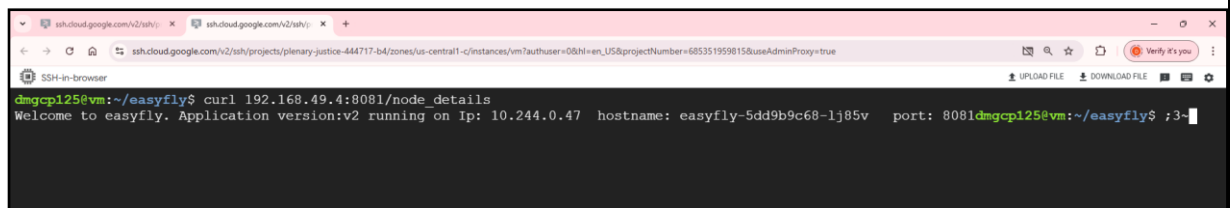
Command: `kubectl get pods`



NAME	READY	STATUS	RESTARTS	AGE
easyfly-5dd9b9c68-lj85v	1/1	Running	0	3m16s
easyfly-5dd9b9c68-nmq1	1/1	Running	1 (117s ago)	3m21s
easyfly-5dd9b9c68-sgxsk	1/1	Running	0	3m16s
easyfly-5dd9b9c68-vtn9q	1/1	Running	0	3m21s
easyfly-5dd9b9c68-vxspm	1/1	Running	0	3m21s
ef-7c86d7dd56-5dmbn	1/1	Running	0	4h2m
mysql-5689f6495f-bd6rm	1/1	Running	0	6m59s
nginx-c95765fd4-7rf8b	1/1	Running	0	5h18m
web-7d676fff59-wz4qm	1/1	Running	0	3h7m

7

Access application



```
dmgcp125@vm:~/easyfly$ curl 192.168.49.4:8081/node_details
Welcome to easyfly. Application version:v2 running on Ip: 10.244.0.47 hostname: easyfly-5dd9b9c68-lj85v port: 8081dmgcp125@vm:~/easyfly$ ;3~
```

Try Other endpoints

1. `curl 192.168.49.4:8081/api-v2/flights/` (GET)
2. `curl 192.168.49.4:8081/api-v2/flights/3` (GET)
3. `curl --header "Content-Type: application/json" --request POST --data '{"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v2/flights/` (POST Request)
4. `curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6,"flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v2/flights/6` (PUT request)
5. `curl --request DELETE 192.168.49.4:8081/api-v2/flights/6` (DELETE)

Activity 11: Deploying Spring Boot app with CPU and Memory Limits

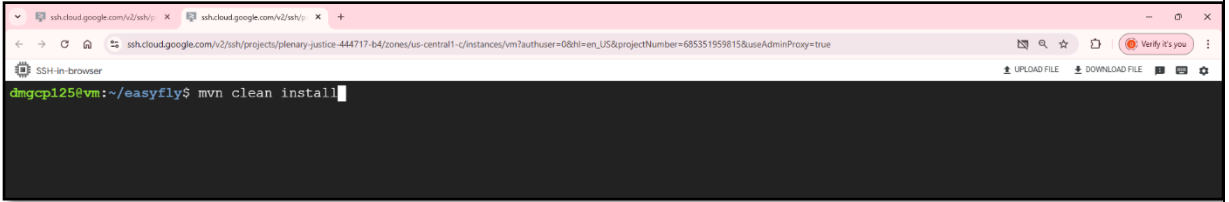
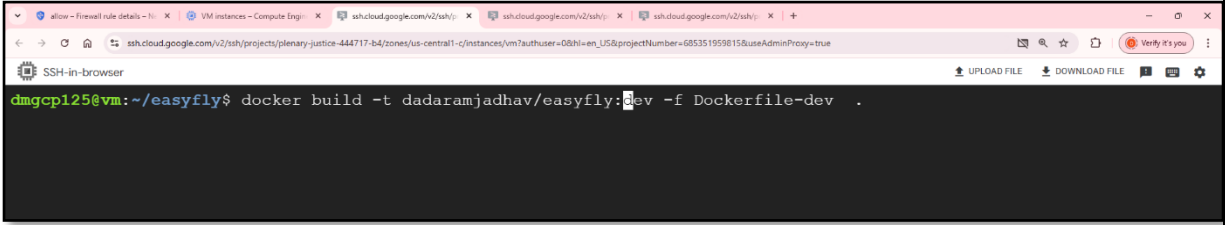
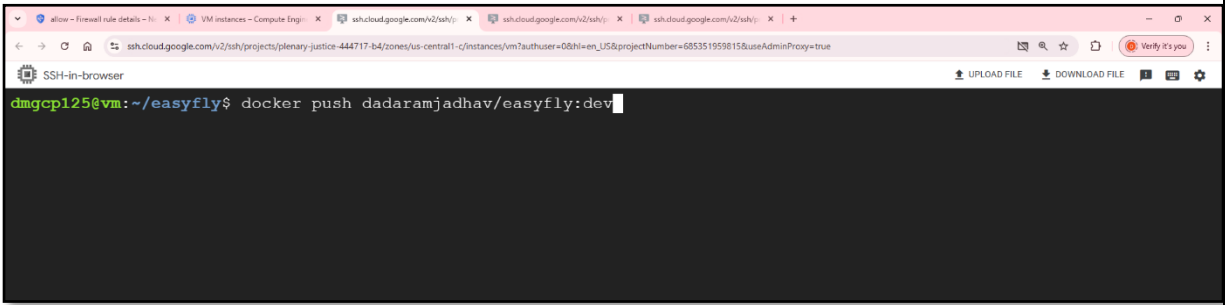
11.1. Prerequisite

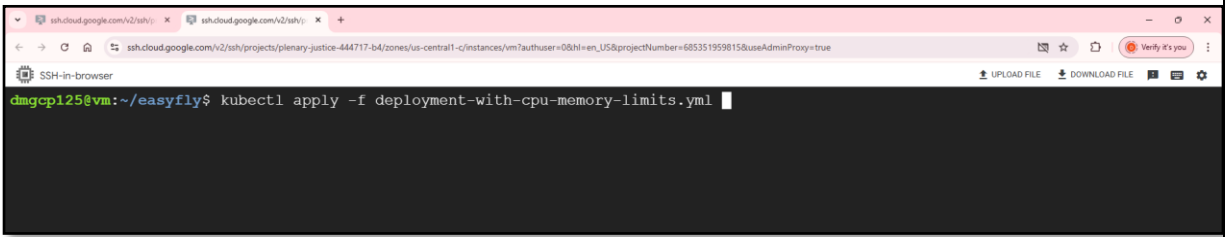
Spring boot application

11.2. Scenario

Deploying spring boot application with CPU and memory limits

11.3. Steps

1	Go to vm
2	Build application Command: mvn clean install 
3	Build docker image Command: docker build -t dadaramjadhav/easyfly:dev -f Dockerfile-dev .  Push image to docker hub registry Command: docker push dadaramjadhav/easyfly:dev 
4	Refer below manifest file apiVersion: apps/v1 kind: Deployment metadata: name: easyfly

	<pre> spec: selector: matchLabels: app: easyfly replicas: 2 template: metadata: labels: app: easyfly spec: containers: - name: easyfly image: dadaramjadhav/easyfly:dev ports: - containerPort: 8081 resources: requests: cpu: 500m memory: 1Gi limits: cpu: 1000m memory: 2Gi --- apiVersion: v1 kind: Service metadata: name: easyfly-svc spec: selector: app: easyfly ports: - protocol: "TCP" port: 8081 targetPort: type: LoadBalancer </pre>
5	Create deployment Command: <code>kubectl apply -f deployment-with-cpu-memory-limits.yml</code> 
6	Get deployment Command: <code>kubectl get deploy</code>

```
ssh.cloud.google.com/v2/ssh/ x ssh.cloud.google.com/v2/ssh/ x +
ssh.cloud.google.com/v2/ssh/projects/planary-justice-444717-b4/zones/us-central1-c/instances/vm7authuser=0&hl=en_US&projectNumber=685351959815&useAdminProxy=true
SSH-in-browser
UPLOAD FILE DOWNLOAD FILE
dmgcp125@vm:~/easyfly$ kubectl get deploy
NAME READY UP-TO-DATE AVAILABLE AGE
easyfly 2/2 2 2 8m49s
dmgcp125@vm:~/easyfly$
```

Describe deployment

Command: kubectl describe deploy <deployment-name>

```
dmgcp125@vm:~/easyfly$ kubectl describe deploy easyfly
Name: easyfly
Namespace: default
CreationTimestamp: Mon, 13 Jan 2025 11:01:37 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=easyfly
Replicas: 2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=easyfly
  Containers:
    easyfly:
      Image: dadaramjadhav/easyfly:dev
      Port: 8081/TCP
      Host Port: 0/TCP
  Limits:
    cpu: 1
    memory: 2Gi
  Requests:
    cpu: 500m
    memory: 1Gi
  Environment: <none>
  Mounts: <none>
  Volumes: <none>
  Node-Selectors: <none>
  Tolerations: <none>
Conditions:
```